Group Project

TITLE : UNIVERSAL CAPABILITY-BASED ROBOTICS ABSTRACTION LAYER FOR DIGITAL TWIN

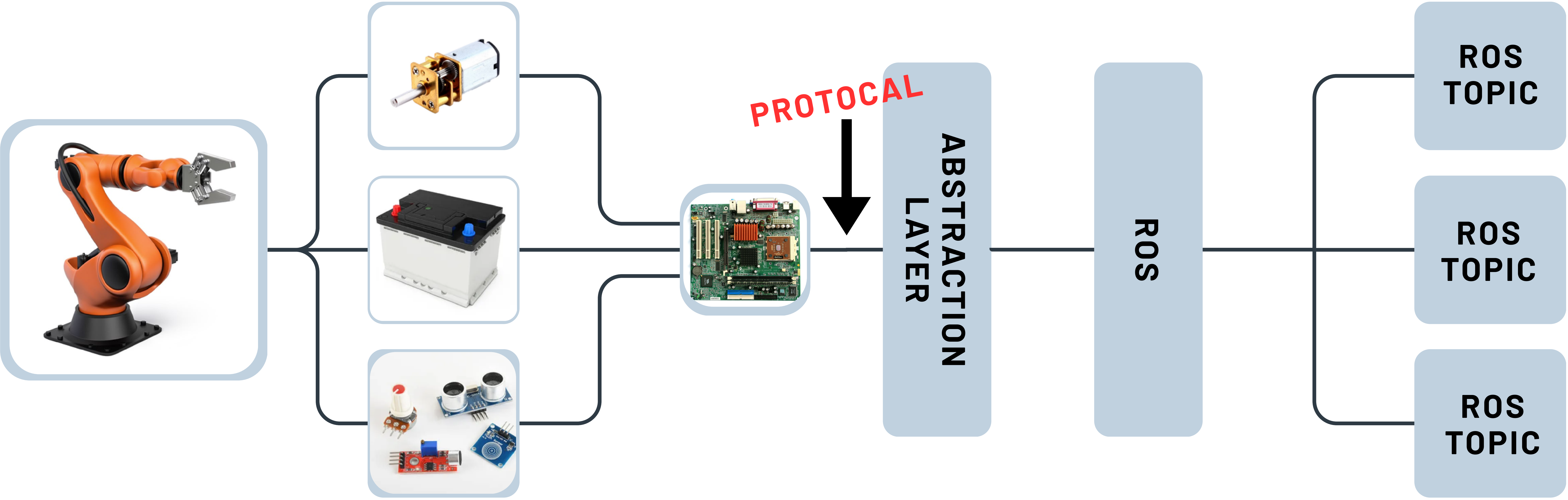
PROBLEM STATEMENT

- Current robotics systems are vendor-specific, tightly coupled, and difficult to integrate.
- **Problems**:
- Every robot uses different APIs/topics/data formats
- Dashboards are manually built
- Digital twin integration is difficult

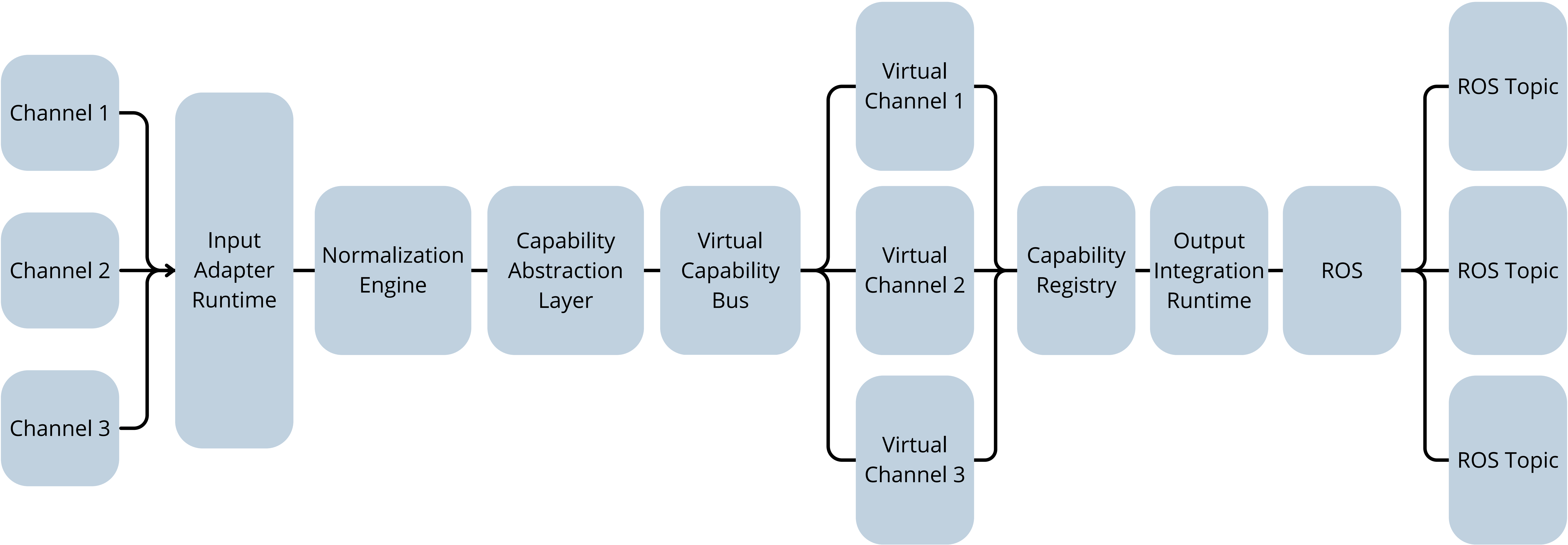
CORE IDEA

- Create a universal capability abstraction layer
- Instead of identifying robotsby model/vendor:
- Robot declares capabilities.

ARCHITECTURE



COMMUNICATION ARCHITECTURE



CAPABILITY SCHEMA

Capability Schema

```json
{
  "robot_id": "R1",
  "capabilities": [
         "movement",
         "camera_stream",
         "battery_monitor"
  ]
}
```

UNIVERSAL CHANNELS

Instead of

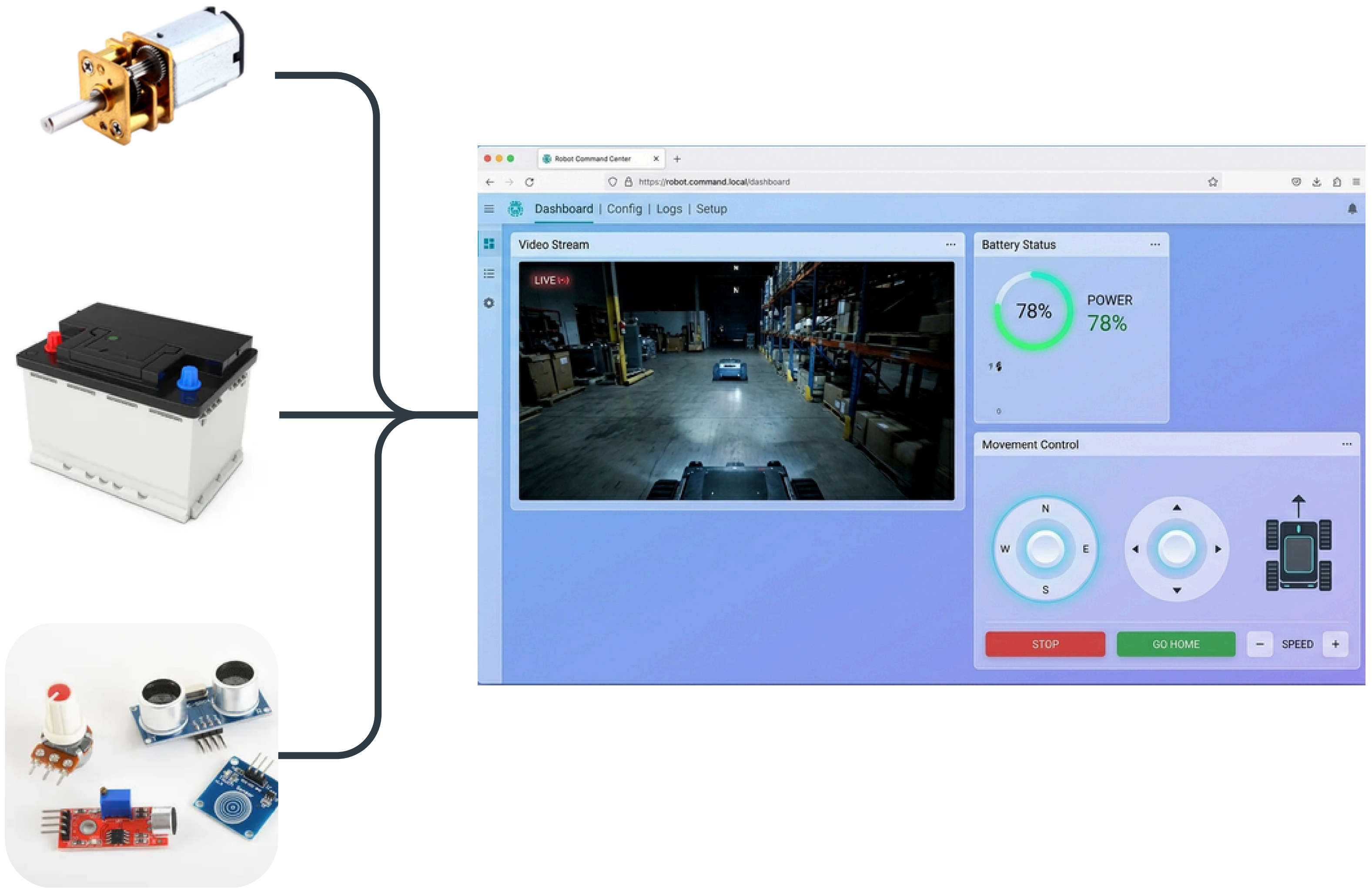
- /robot1_battery
- /robot2_battery

We use

- /capabilities/battery
- /capabilities/camera
- /capabilities/movem

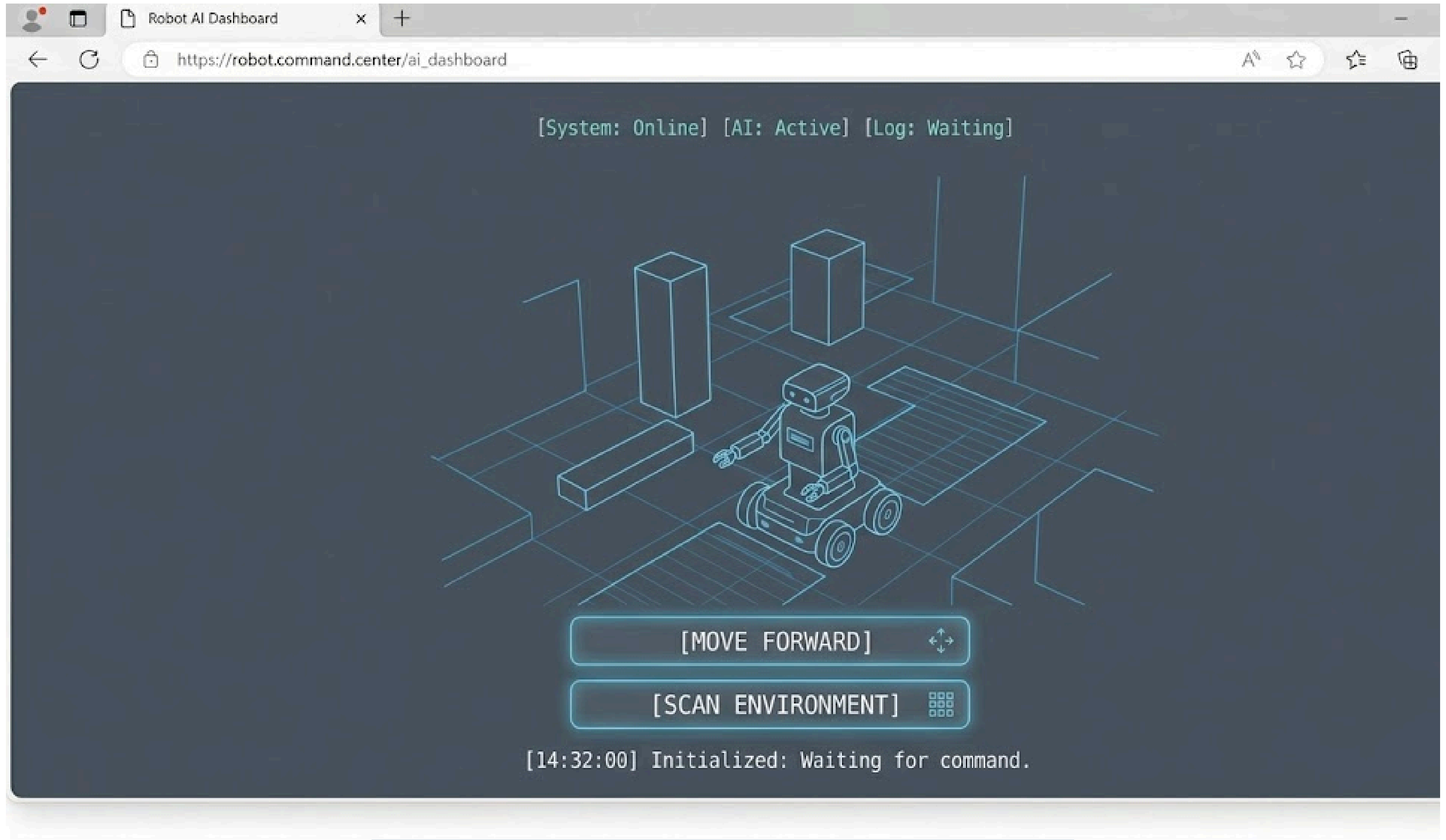
AUTO DASHBOARD GENERATION

- Dashboard queries capability registry.
- If capability exists
    - battery → show battery widget
    - camera → show video stream
    - movement → show joystick controls
- UI generates automatically.



AI INTEGRATION

- AI agents interact with capabilities instead of robot specific code.



MARKET POTENTIAL

- **Applications**
    - Industrial robotics
    - Smart factories
    - Warehouses
    - Service robots
    - Defense systems
    - Healthcare robotics
    - Research labs

MARKET POTENTIAL

- **Applications**
  - Industrial robotics
  - Smart factories
  - Warehouses
  - Service robots
  - Defense systems
  - Healthcare robotics
  - Research labs

REFERENCES

1. **A multi-robot collaborative manipulation framework for dynamic and obstacle-dense environments: integration of deep learning for real-time task execution - PMC** https://share.google/aczHZWtgwzcvmsdID
2. **Personalising healthcare with connected digital twins | The Alan Turing Institute** https://share.google/ugxBQgK7hUsw2fdZT
3. **Evolution 6.0: Evolving Robotic Capabilities Through Generative Design** https://share.google/Q336dAnlHmzzrw1HC
4. **Capability-based Frameworks for Industrial Robot Skills: a Survey** https://share.google/2QyPHZMAVz7Q7nU6i
5. **Capability-oriented robot architecture for maritime autonomy - ScienceDirect** https://share.google/aczHZWtgwzcvmsdID

CONCLUSION

The proposed system acts as a **USB for Robotics** enabling plug-and-play interoperability between robots, AI modules, sensors, and automation systems through a universal intelligent **capability integration layer.**

Thank You

INPUT ADAPTER RUNTIME

This layer:

- accepts vendor-specific protocols
- handles transport
- manages device communication

Examples:

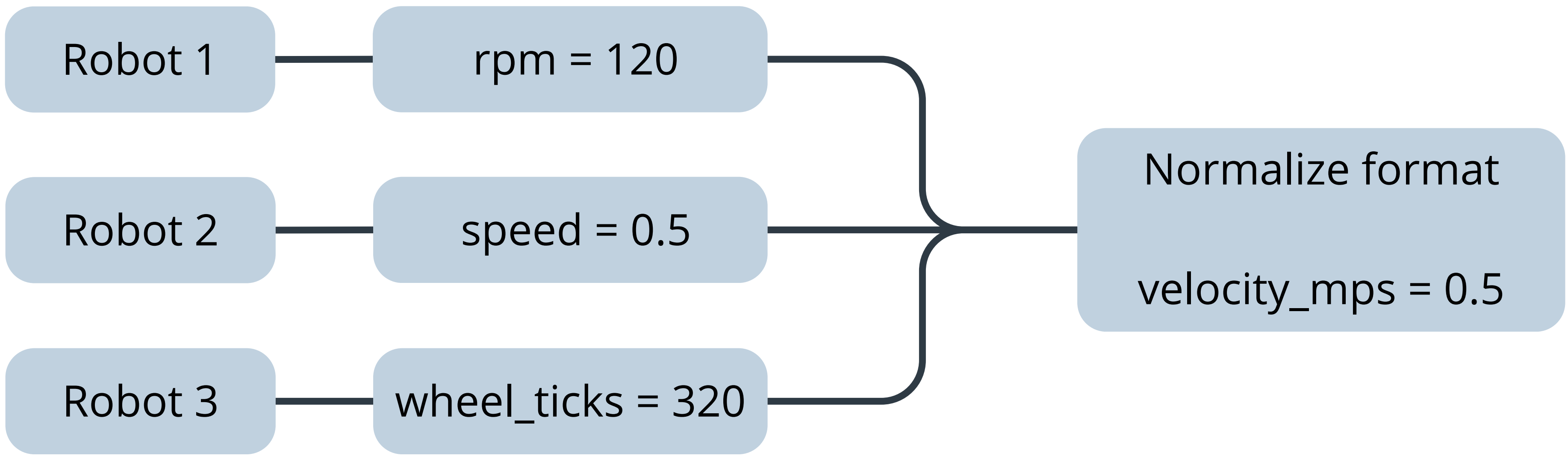
- MQTT
- TCP
- Serial
- Modbus
- CAN bus
- WebSocket

NORMALIZATION ENGINE

This layer:

- handle data standardization
- handles field mapping
- manages unit conversion

Examples:

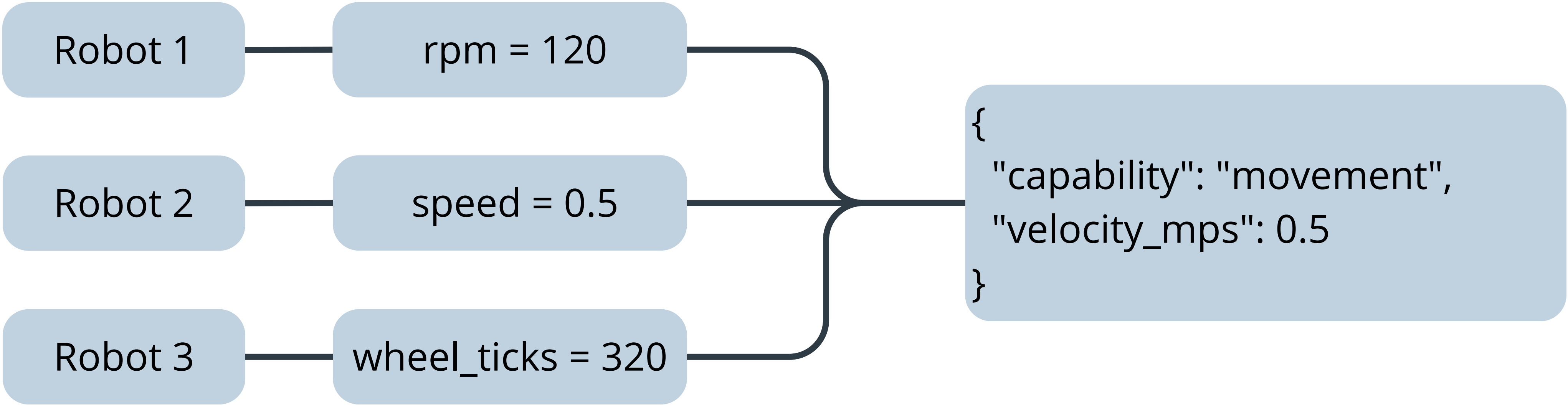


CAPABILITY ABSTRACTION LAYER

This layer:

- converts normalized hardware data into semantic capabilities
- maps low-level telemetry into high-level functional meaning
- enables interoperability between different robot vendors
- abstracts robot-specific implementations

Examples:



Note

- Without abstraction:

Every robot requires custom software.

- With abstraction:

Any robot exposing capabilities becomes universally usable.

VIRTUAL CAPABILITY BUS

This layer:

- transports standardized capability events
- decouples producers from consumers
- enables multi-platform integration

VIRTUAL CHANNELS

This layer:

- transports standardized capability events
- decouples producers from consumers
- enables multi-platform integration

CAPABILITY REGISTRY

This layer:

- the brain of orchestration

Stores:

- robot metadata
- schemas
- capabilities
- active endpoints
- dependencies
- health state

Enables:

- auto-discovery
- service binding
- AI orchestration

OUTPUT INTEGRATION RUNTIME

This layer:

- converts capability streams into target platforms
- generates ROS topics
- sent feed to dashboards

Enables:

- auto-discovery
- service binding
- AI orchestration

ROS LAYER

This layer:

- a downstream consumer

Can be converted to :

- ROS2
- Unity
- Unreal
- Web apps
- Digital twins
- AI agents
- cloud systems

LAYERS V/S RESPONSIBILITY

Layer	Responsibility
Input Runtime	Communication
Normalization	Data cleanup
Capability Layer	Semantic meaning
Capability Bus	Distribution
Registry	Discovery/intelligence
Output Runtime	Platform adaptation

APPROACHING CONCEPTS FROM

System	Similarity
Apache Kafka	Event bus
DDS	Real-time middleware
OPC UA	Industrial abstraction
Kubernetes	Service discovery
ROS2	Robotics communication
MQTT brokers	Messaging
Digital Twin platforms	State synchronization

FUTURE ASPEST

1. Capability Dependency Resolver → Because some capabilities require others.
2. Policy → Rules

PATENTABLE COMPONENTS

- **Capability Negotiation Engine**

Automatically discovers, validates, and matches robot capabilities with compatible services in the system.
Patentable since: Provides automatic interoperability of capabilities at run-time, without any integration with specific robots.

- **Universal Robotics Capability Bus**

Standardized communication mechanism for interoperable exchange of capabilities data by robots.
Patentable since: Builds a universal semantic middleware layer irrespective of any particular robot or its protocol.

- **Dynamic Capability Registry**

Runtime database maintaining robot capabilities, dependency graphs, capability schemas, and associated services.
Patentable since: Helps discover robotic services and automate their orchestration in heterogeneous systems.

- **Automated Dashboard Generation System**

Automatic creation of adaptive dashboards for robots using detected capabilities and telemetry streams.
Patentable since: Generates user interfaces dynamically, without any need for manual UI development.

- **AI-Oriented Capability Interface**

Enables AI programs to control robots by sending standard capability-related semantic commands.
Patentable since: Makes a robot hardware independent from the perspective of AI programs.

- **Vendor-Independent Robot Adapter Layer**

Translates vendor-dependent robot commands into generic capabilities supported by all robots.
Patentable since: Helps create vendor-independent robot capabilities without any modifications to underlying robots.